Principles of Distributed Systems

第8周：容错

Week 8: Fault Tolerance

By: Fahad Sabah

计算机学院,
北京工业大学,中国
College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn

容错
Fault Tolerance

- ❑ Basic concepts
- ❑ Process resilience
- ❑ Reliable client-server communication
- ❑ Reliable group communication
- ❑ Distributed commit
- ❑ Recovery

- ❑ 基本概念
- ❑ 过程韧性
- ❑ 可靠的客户端-服务器通信
- ❑ 可靠的群体通信
- ❑ 分布式提交
- ❑ 恢复

简介
Introduction

❑ **Dependability**

A component provides services to clients. To provide services, the component may require the services from other components ⇒ a component may depend on some other component.

❑ **Specifically**

A component C depends on C∗ if the correctness of C's behavior depends on the correctness of C∗'s behavior.

Note: components are processes or channels.

- ❑ **Availability** Readiness for usage
- ❑ **Reliability** Continuity of service delivery
- ❑ **Safety** Very low probability of disasters
- ❑ **Maintainability** How easy can a failed system be repaired

❑ **可靠性**

组件为客户提供服务。为了提供服务，组件可能需要其他组件的服务，⇒组件可能依赖于其他组件。

❑ **具体来说**

如果C行为的正确性依赖于C∗行为的正确性，则C依赖于C∗。

注意：组件是进程或通道。

- ❑ 可用性 使用准备情况
- ❑ 可靠性 服务连续性
- ❑ 安全性 灾难发生概率极低
- ❑ 可维护性 故障系统修复有多容易

简介
Introduction

Basic differences

Failure: When a component is not living up to its specifications, a failure occurs

Error: That part of a component's state that can lead to a failure

Fault: The cause of an error

What to do about faults

- **Fault prevention:** prevent the occurrence of a fault
- **Fault tolerance:** build a component such that it can mask the presence of faults
- **Fault removal:** reduce presence, number, seriousness of faults
- **Fault forecasting:** estimate present number, future incidence, and consequences of faults

基本区别

故障：当组件未达到其规格时，会发生故障

错误：组件状态中可能导致故障的部分

故障：错误的原因

如何处理故障

故障预防：防止故障发生

容错：构建一个能够掩盖故障存在的组件

故障消除：减少故障的存在、数量和严重性

故障预测：估算当前故障数量、未来发生率及故障后果

简介
Introduction

Failure models/Failure semantics

- **Crash failures:** Component halts, but behaves correctly before halting
- **Omission failures:** Component fails to respond
- **Timing failures:** Output is correct, but lies outside a specified real-time interval (performance failures: too slow)
- **Response failures:** Output is incorrect (but can at least not be accounted to another component)
- **Value failure:** Wrong value is produced
- **State transition failure:** Execution of component brings it into a wrong state
- **Arbitrary failures:** Component produces arbitrary output and be subject to arbitrary timing failures

失效模型/失效语义

- **崩溃故障：**组件会停止，但在停止前表现正常
- **遗漏失败：**组件未能响应
- **时序失效：**输出正确，但超出指定的实时区间（性能失误：过慢）
- **响应失败：**输出错误（但至少不能归因于其他组件）
- **价值失败：**产生错误的价值
- **状态转换失败：**组件的执行使其进入错误状态
- **任意故障：**组件产生任意输出，并受到任意时序故障的影响

简介
Introduction

❑ **Crash failures**

Problem

Clients cannot distinguish between a crashed component and one that is just a bit slow

Consider a server from which a client is expecting output

❑ Is the server perhaps exhibiting timing or omission failures?

❑ Is the channel between client and server faulty?

Assumptions we can make

❑ **Fail-silent:** The component exhibits omission or crash failures; clients cannot tell what went wrong

❑ **Fail-stop:** The component exhibits crash failures, but its failure can be detected (either through announcement or timeouts)

❑ **Fail-safe:** The component exhibits arbitrary, but benign failures (they can't do any harm)

坠机故障

问题

客户端无法区分崩溃的组件和只是稍微慢的组件

考虑一个服务器，客户端期望从中输出

❑ 服务器是否存在时间错乱或遗漏故障?

❑ 客户端和服务器之间的通道有问题吗?

我们可以做出的假设

❑ **失效静音：**该组件出现遗漏或崩溃故障;客户无法判断哪里出了问题

❑ **故障停止：**组件表现出崩溃故障，但其故障可以通过公告或超时被检测到

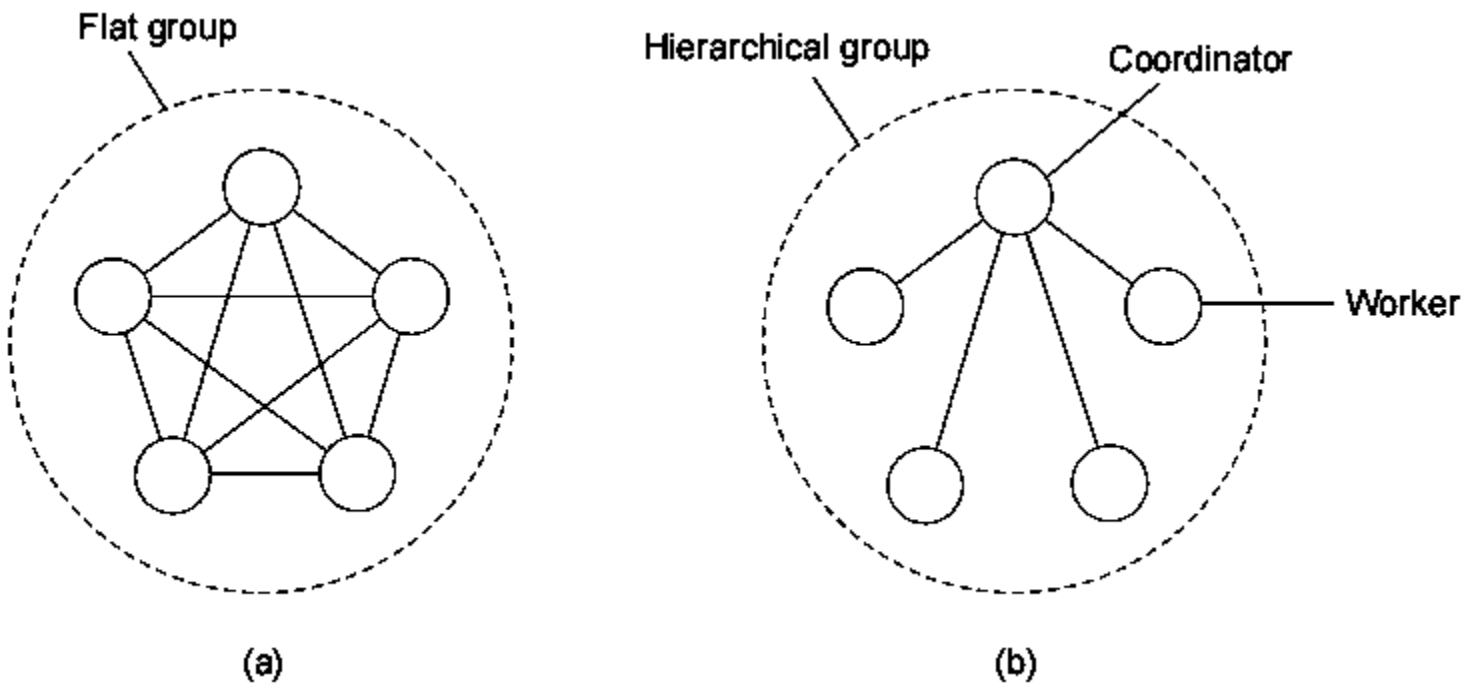
❑ **故障保护：**该部件表现出一些任意但无害的故障（它们不会造成任何伤害）

工艺韧性
Process Resilience

- **Process resilience**
- Protect yourself against faulty processes by replicating and distributing computations in a group.
  - **Flat groups:** Good for fault tolerance as information exchange immediately occurs with all group members; however, may impose more overhead as control is completely distributed (hard to implement).
  - **Hierarchical groups:** All communication through a single coordinator⇒ not really fault tolerant and scalable, but relatively easy to implement.

- **过程韧性**
- 通过在组中复制和分发计算来保护自己免受错误进程的影响。
  - **平面组：**适合容错，因为信息交换会立即与所有组成员进行;然而，由于控制完全分散，可能会带来更多开销（实现困难）。
  - **层级组：**所有通信都通过单一协调员进行⇒不具备容错性和可扩展性，但相对容易实现。

工艺韧性
Process Resilience



工艺韧性
Process Resilience

Groups and failure masking

K-fault tolerant group

When a group can mask any k concurrent member failures (k is called degree of fault tolerance).

How large does a k-fault tolerant group need to be?

- ❑ Assume crash/performance failure semantics ⇒ a total of k +1 members are needed to survive k member failures.
- ❑ Assume arbitrary failure semantics, and group output defined by voting ⇒ a total of 2k +1 members are needed to survive k member failures.

Assumption

All members are identical, and process all input in the same order ⇒ only then are we sure that they do exactly the same thing.

工艺韧性
Process Resilience

Groups and failure masking

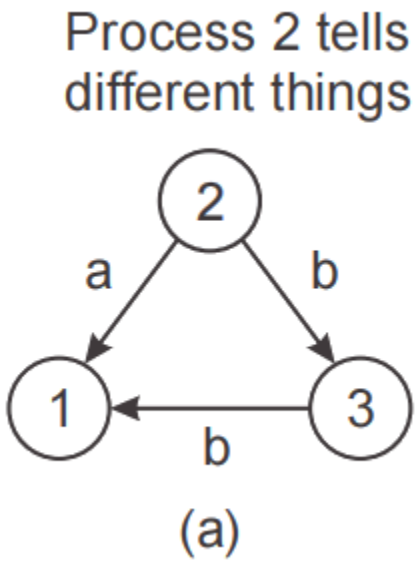
Scenario

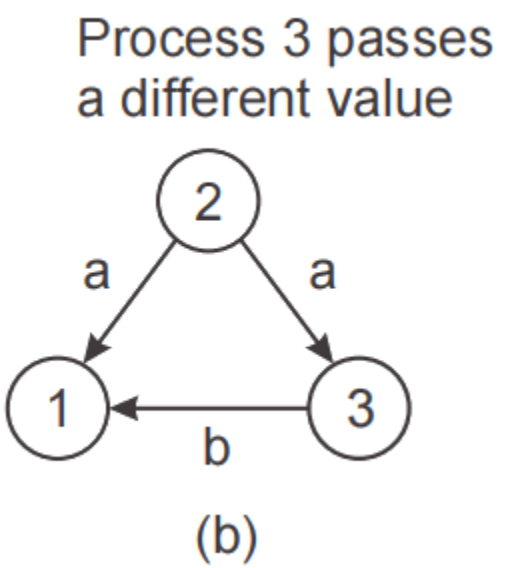
Group members are not identical, i.e., we have a distributed computation ⇒ Nonfaulty group members should reach agreement on the same value.

Simple Real-Life Analogy

Imagine 3 friends deciding where to eat.
Friend 2 tells Friend 1: “Let’s eat pizza.”
But tells Friend 3: “Let’s eat burgers.”

Now the group becomes confused because different people received different instructions.





工艺韧性
Process Resilience

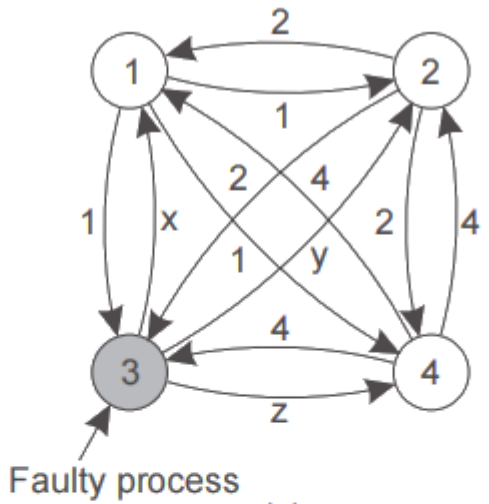
Groups and failure masking

Scenario

Assuming arbitrary failure semantics, we need 3k +1 group members to survive the attacks of k faulty members. This is also known as **Byzantine failures**.

Essence

We are trying to reach a majority vote among the group of loyalists, in the presence of k traitors ⇒ need 2k +1 loyalists.



(a)

1 Got(1, 2, x, 4)
2 Got(1, 2, y, 4)
3 Got(1, 2, 3, 4)
4 Got(1, 2, z, 4)

(b)

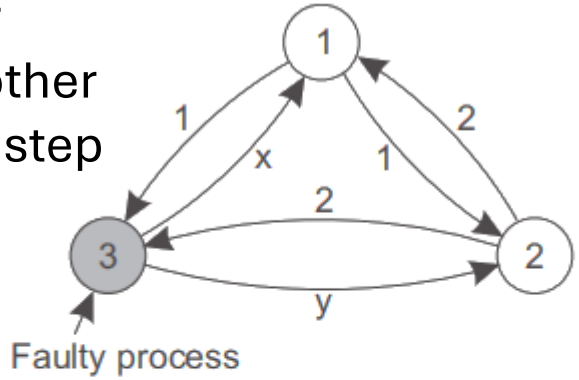
1 Got	2 Got	4 Got
(1, 2, y, 4)	(1, 2, x, 4)	(1, 2, x, 4)
(a, b, c, d)	(e, f, g, h)	(1, 2, y, 4)
(1, 2, z, 4)	(1, 2, z, 4)	(i, j, k, l)

(c)

工艺韧性
Process Resilience

Groups and failure masking

(a) what they send to each other
(b) what each one got from the other
(c) what each one got in second step



(a)

1	Got(1, 2, x)
2	Got(1, 2, y)
3	Got(1, 2, 3)

(b)

1 Got	2 Got
(1, 2, y)	(1, 2, x)
(a, b, c)	(d, e, f)

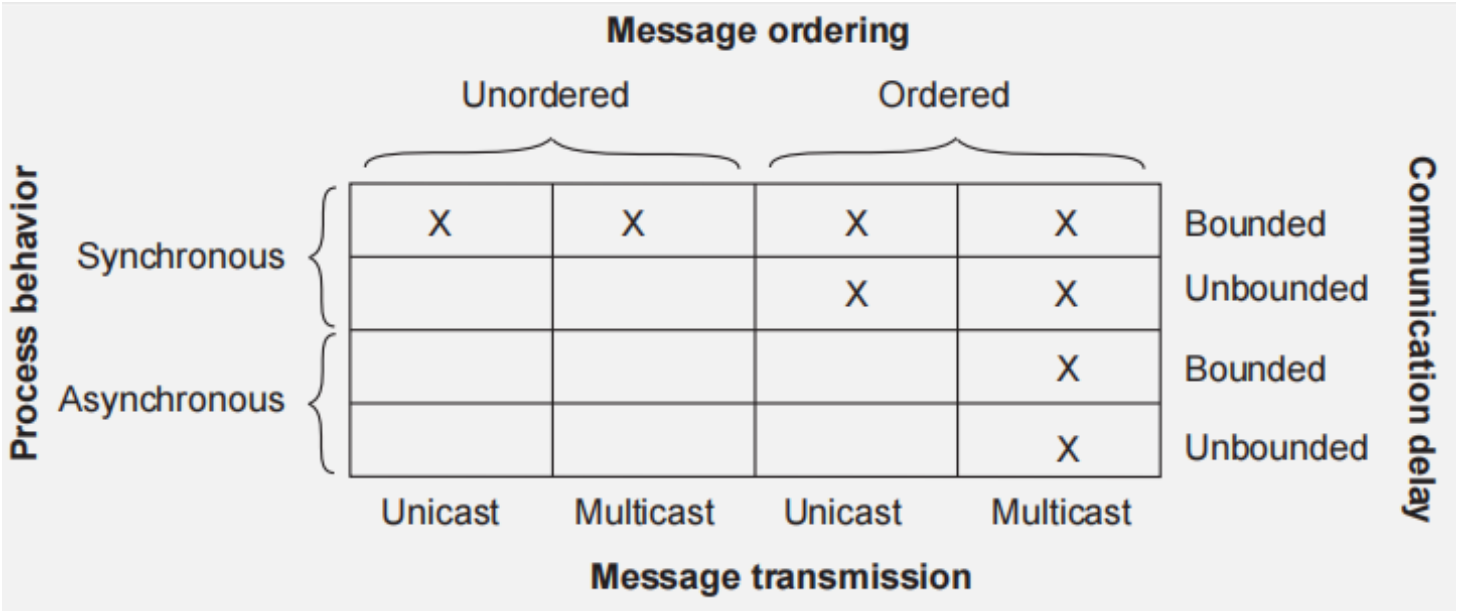
(c)

工艺韧性
Process Resilience

Groups and failure masking

Issue

What are the necessary conditions for reaching agreement?



Process: Synchronous ⇒ operate in lockstep

Delays: Are delays on communication bounded?

Ordering: Are messages delivered in the order they were sent?

Transmission: Are messages sent one-by-one, or multicast?

工艺韧性
Process Resilience

Failure detection

We detect failures through timeout mechanisms

- Setting timeouts properly is very difficult and application dependent
- You cannot distinguish process failures from network failures
- We need to consider failure notification throughout the system:
  - Gossiping (i.e., proactively disseminate a failure detection)
  - On failure detection, pretend you failed as well

故障检测

我们通过超时机制检测故障

- 正确设置超时非常困难且依赖应用程序
- 你无法区分进程故障和网络故障
- 我们需要考虑整个系统的故障通知：
  - 八卦（即主动传播故障检测结果）
  - 在故障检测时，假装你也失败了

可靠的通信
Reliable communication

So far
Concentrated on process resilience (by means of process groups).
What about reliable communication channels?

Error detection
- ❑ Framing of packets to allow for bit error detection
- ❑ Use of frame numbering to detect packet loss

Error correction
- ❑ Add so much redundancy that corrupted packets can be automatically corrected
- ❑ Request retransmission of lost, or last N packets

到目前为止
重点关注过程韧性（通过流程组）。
可靠的沟通渠道呢？

错误检测
数据包的帧以实现比特错误检测
使用帧编号检测丢包

纠错
增加大量冗余，损坏的数据包可以自动纠正
请求重传丢失或最近N个数据包

可靠的通信
Reliable communication

Reliable RPC

RPC communication: What can go wrong?

1: Client cannot locate server

2: Client request is lost

3: Server crashes

4: Server response is lost

5: Client crashes

RPC communication: Solutions

1: Relatively simple – just report back to client

2: Just resend message

可靠的RPC

RPC沟通：会出什么问题?

1：客户端无法定位服务器

2：客户端请求丢失

3：服务器崩溃

4：服务器响应丢失

5：客户端崩溃

RPC通信：解决方案

1：相对简单——只需向客户汇报

2：直接重新发送消息

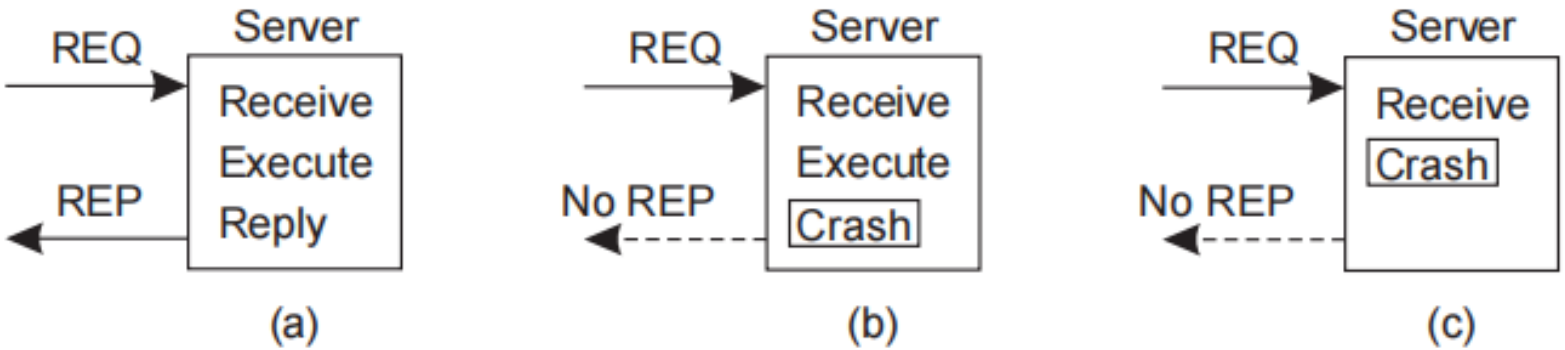
可靠的通信
Reliable communication

Reliable RPC

RPC communication: Solutions

Server crashes

3: Server crashes are harder as you don't what it had already done:



可靠的通信
Reliable communication

Reliable RPC

We need to decide on what we expect from the server

- At-least-once-semantics: The server guarantees it will carry out an operation at least once, no matter what.
- At-most-once-semantics: The server guarantees it will carry out an operation at most once.

可靠的RPC

我们需要决定对服务器的期望

- 至少一次语义：服务器保证无论如何至少执行一次操作。
- 最多一次语义：服务器保证最多执行一次操作。

可靠的通信
Reliable communication

RPC communication: Solutions

Server response is lost

4: Detecting lost replies can be hard, because it can also be that the server had crashed. You don't know whether the server has carried out the operation

Solution: None, except that you can try to make your operations idempotent: repeatable without any harm done if it happened to be carried out before.

可靠的通信
Reliable communication

RPC communication: Solutions

Client crashes

5: Problem: The server is doing work and holding resources for nothing (called doing an orphan computation).

- ❑ Orphan is killed (or rolled back) by client when it reboots
- ❑ Broadcast new epoch number when recovering ⇒ servers kill orphans
- ❑ Require computations to complete in a T time units. Old ones are simply removed.

RPC通信：解决方案
客户端崩溃

5：问题：服务器无偿地工作并占用资源（称为孤儿计算）。

- ❑Orphan在重启时被客户端杀死（或回滚）
- ❑恢复时广播新的纪元号⇒服务器会杀死孤儿
- ❑要求计算在T时间单位内完成。旧的会被直接移除。

可靠的群组通信
Reliable Group Communication

Reliable multicasting

Basic model

We have a multicast channel c with two (possibly overlapping) groups:

- ❑The sender group SND(c) of processes that submit messages to channel c
- ❑The receiver group RCV(c) of processes that can receive messages from channel c

Simple reliability: If process P ∈ RCV(c) at the time message m was submitted to c, and P does not leave RCV(c), m should be delivered to P

Atomic multicast: How can we ensure that a message m submitted to channel c is delivered to process P ∈ RCV(c) only if m is delivered to all members of RCV(c)

可靠的群组通信
Reliable Group Communication

Reliable multicasting

Observation

If we can stick to a local-area network, reliable multicasting is “easy”

Principle

Let the sender log messages submitted to channel c:

- If P sends message m, m is stored in a history buffer
- Each receiver acknowledges the receipt of m, or requests retransmission at P when noticing message lost
- Sender P removes m from history buffer when everyone has acknowledged receipt

可靠的多播

观察

如果我们能坚持局域网，可靠的多播“很容易”

原理

让发送方记录向信道c提交的消息：

- 如果 P 发送消息 m，m 会被存储在历史缓冲区中
- 每个接收端在发现消息丢失时都会确认收到m，或在P处请求重传
- 当所有人都确认收到后，发送方P会从历史缓冲区移除m。

可靠的群组通信
Reliable Group Communication

Scalable reliable multicasting: Feedback suppression

Basic idea

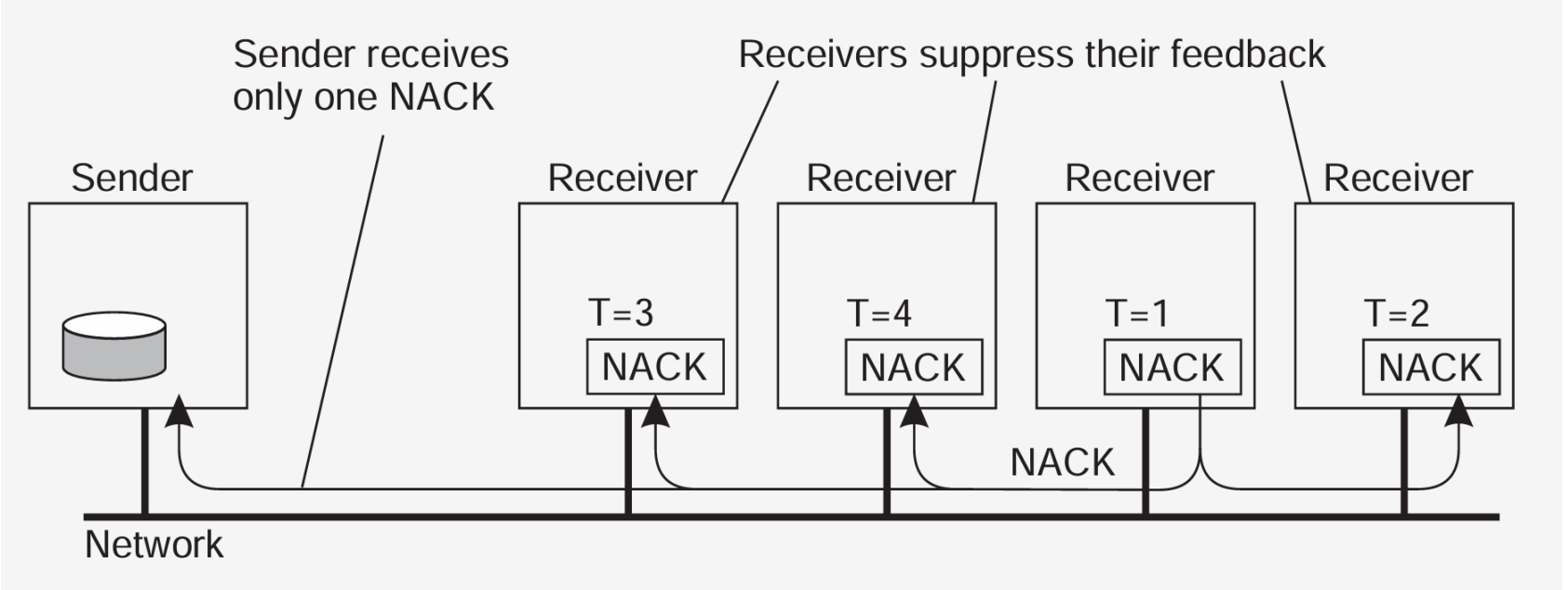
Let a process P suppress its own feedback when it notices another process Q is already asking for a retransmission

Assumptions

- All receivers listen to a common feedback channel to which feedback messages are submitted
- Process P schedules its own feedback message randomly, and suppresses it when observing another feedback message

可靠的群组通信
Reliable Group Communication

Scalable reliable multicasting: Feedback suppression



可靠的群组通信
Reliable Group Communication

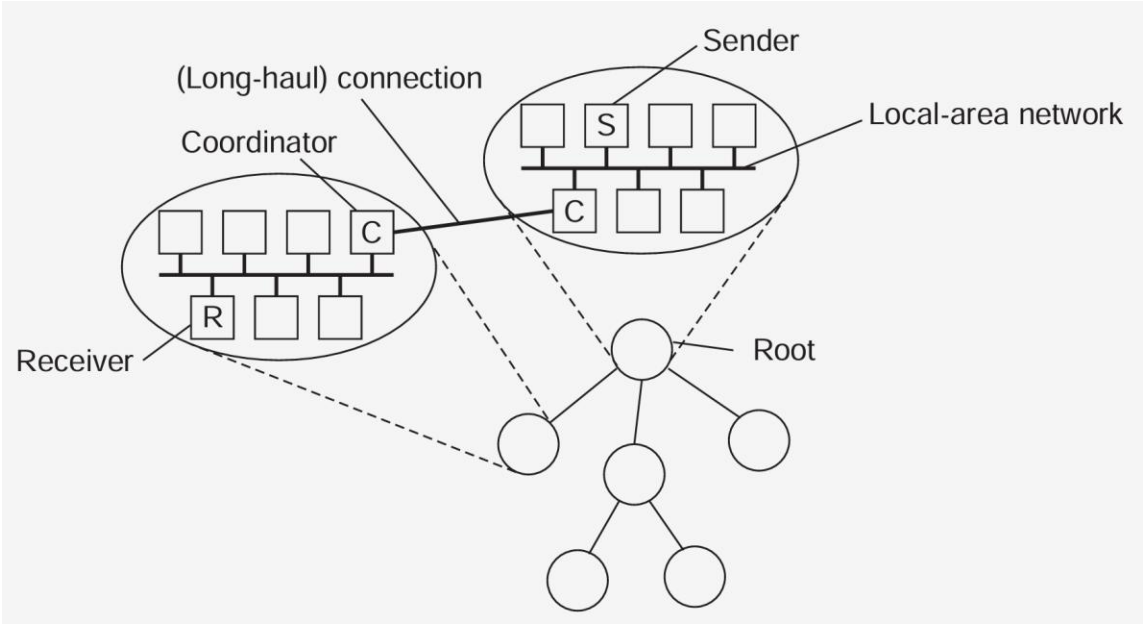
Scalable reliable multicasting: Hierarchical solutions

Construct a hierarchical feedback channel in which all submitted messages are sent only to the root. Intermediate nodes aggregate
feedback messages before passing them on.

Observation
Intermediate nodes can easily be used for retransmission purposes

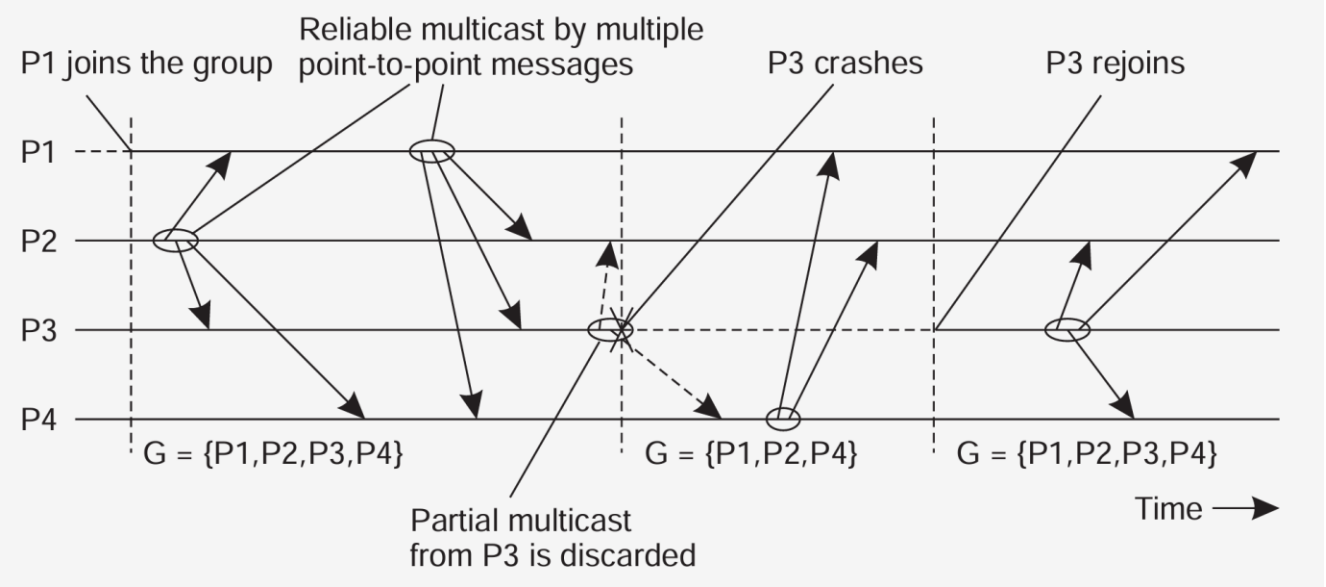
可靠的群组通信
Reliable Group Communication

Scalable reliable multicasting: Hierarchical solutions



可靠的群组通信
Reliable Group Communication

Atomic multicast

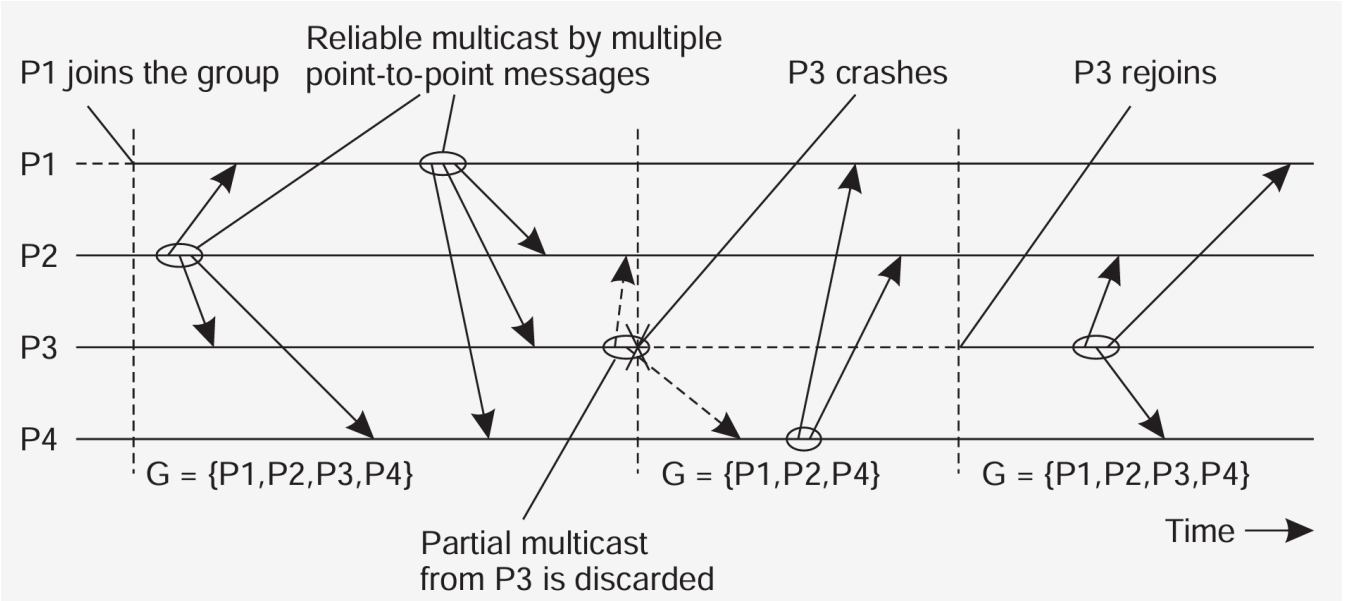


Idea

Formulate reliable multicasting in the presence of process failures in terms of process groups and changes to group membership.

可靠的群组通信
Reliable Group Communication

Atomic multicast



A message is delivered only to the nonfaulty members of the current group.
All members should agree on the current group membership ⇒Virtually synchronous multicast.

可靠的群组通信
Reliable Group Communication

Virtual synchrony

Essence

We consider views V ⊆RCV(c)∪SND(c)

Principle

Processes are added or deleted from a view V through view changes
to V∗; a view change is to be executed locally by each P ∈V ∩V∗

(1) For each consistent state, there is a unique view on which all its
members agree. Note: implies that all nonfaulty processes see all view changes in the same order

可靠的群组通信
Reliable Group Communication

Virtual synchrony

Principle (cnt'd)

(2) If message m is sent to V before a view change vc to V∗, then either all P ∈ V that execute vc receive m, or no processes P ∈V that execute vc receive m. Note: all nonfaulty members in the same view get to see the same set of multicast messages.

(3) A message sent to view V can be delivered only to processes in V, and is discarded by successive views

Definition

A reliable multicast algorithm satisfying (1)–(3) is virtually synchronous

可靠的群组通信
Reliable Group Communication

Virtual synchrony

How it works

1. A sender to a view V need not be member of V
2. If a sender S ∈V crashes, its multicast message m is flushed before S is removed from V: m will never be delivered after the point that S ∈ V
3. Note: Messages from S may still be delivered to all, or none (nonfaulty) processes in V before they all agree on a new view to which S does not belong
4. If a receiver P fails, a message m may be lost but can be recovered as we know exactly what has been received in V.

Alternatively, we may decide to deliver m to members in V −{P}

可靠的群组通信
Reliable Group Communication

Virtual synchrony

Observation
Virtually synchronous behavior can be seen independent from the ordering of message delivery. The only issue is that messages are delivered to an agreed upon group of receivers.

可靠的群组通信
Reliable Group Communication

Virtual synchrony implementation

- The current view is known at each P by means of a delivery list dest[P]
- If P ∈dest[Q] then Q ∈dest[P]
- Messages received by P are queued in queue[P]
- If P fails, the group view must change, but not before all messages from P have been flushed
- Each P attaches a (stepwise increasing) timestamp with each message it sends
- Assume FIFO-ordered delivery; the highest numbered message from Q that has been received by P is recorded in rcvd[P][Q]
- The vector rcvd[P][] is sent (as a control message) to all members in dest[P]
- Each P records rcvd[Q][] in remote[P][Q]

可靠的群组通信
Reliable Group Communication

Virtual synchrony implementation

Observation

remote[P][Q] shows what P knows about message arrival at Q

1	2	3	1	5
2	2	2	2	4
3	3	1	4	5
4	4	2	2	4
min	2	1	1	4

可靠的群组通信
Reliable Group Communication

Virtual synchrony implementation

Principle

- A message is stable if it has been received by all Q ∈ dest[P] (shown as the min vector)
- Stable messages can be delivered to the next layer (which may deal with ordering). Note: Causal message delivery comes for free
- As soon as all messages from the faulty process have been flushed, that process can be removed from the (local) views

可靠的群组通信
Reliable Group Communication

Virtual synchrony implementation

Remains

What if a sender P failed and not all its messages made it to the nonfaulty members of the current view?

Solution

Select a coordinator which has all (unstable) messages from P, and forward those to the other group members.

Note

Member failure is assumed to be detected and subsequently multicast to the current view as a view change. That view change will not be carried out before all messages in the current view have been delivered.

分布式提交
Distributed Commit

Distributed commit

- Two-phase commit
- Three-phase commit

Essential issue

Given a computation distributed across a process group, how can we ensure that either all processes commit to the final result, or none of them do (atomicity)?

分布式提交

- 两阶段提交
- 三阶段提交

核心问题

给定分布在一个进程组中的计算，我们如何确保所有进程都承诺最终结果，或者都不承诺（原子性）？

分布式提交
Distributed Commit

Two-phase commit

Model

The client who initiated the computation acts as coordinator; processes required to commit are the participants

- Phase 1a: Coordinator sends vote-request to participants (also called a pre-write)
- Phase 1b: When participant receives vote-request it returns either vote-commit or vote-abort to coordinator. If it sends vote-abort, it aborts its local computation
- Phase 2a: Coordinator collects all votes; if all are vote-commit, it sends global-commit to all participants, otherwise it sends global-abort
- Phase 2b: Each participant waits for global-commit or global-abort and handles accordingly.

两阶段提交

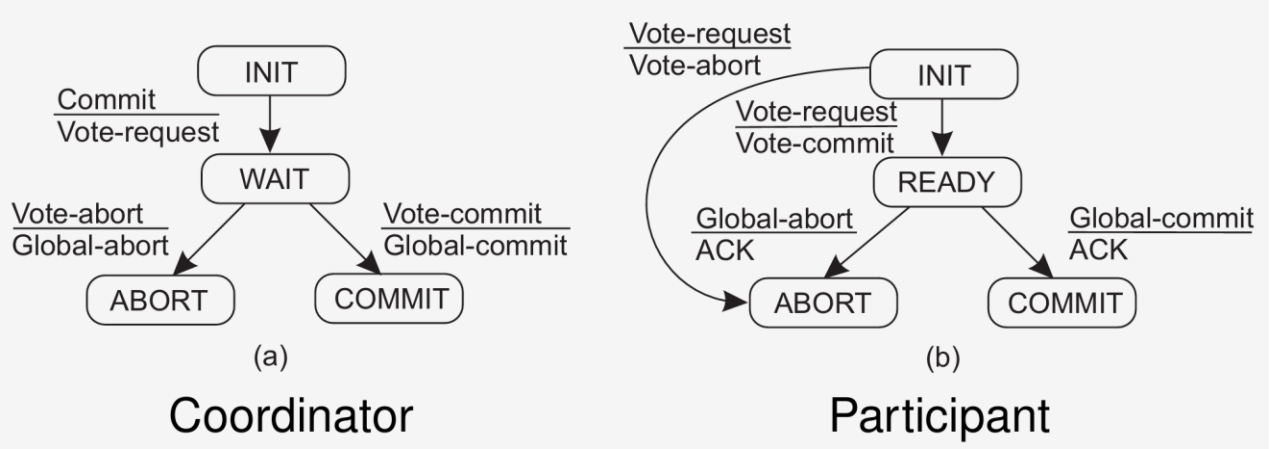
模型

发起计算的客户端充当协调者;承诺所需的流程是参与者

- 第一阶段a：协调员向参与者发送投票请求（也称为预写）
- 阶段1b：当参与者收到投票请求时，会将投票承诺或投票中止返回给协调者。如果发送投票中止，则会中止其局部计算
- 第二阶段a：协调员收集所有选票;如果所有参与者都投投提交，则发送全局提交给所有参与者，否则发送全局中止
- 第二阶段b：每位参与者等待全局提交或全局中止，并相应处理。

分布式提交
Distributed Commit

Two-phase commit



分布式提交
Distributed Commit

2PC– Failing participant

Scenario

Participant crashes in state S, and recovers to S

Initial state: No problem: participant was unaware of protocol

Ready state: Participant is waiting to either commit or abort. After recovery, participant needs to know which state transition it should make ⇒log the coordinator's decision

Abort state: Merely make entry into abort state idempotent, e.g., removing the workspace of results

Commit state: Also make entry into commit state idempotent, e.g., copying workspace to storage.

Observation

When distributed commit is required, having participants use temporary workspaces to keep their results allows for simple recovery in the presence of failures.

分布式提交
Distributed Commit

2PC–Failing participant

Alternative

When a recovery is needed to READY state, check state of other participants ⇒no need to log coordinator's decision.

Recovering participant P contacts another participant Q

State of Q	Action by P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

Result

If all participants are in the READY state, the protocol blocks. Apparently, the coordinator is failing.

Note: The protocol prescribes that we need the decision from the coordinator.

可靠的通信
Distributed Commit

2PC– Failing coordinator

Observation

The real problem lies in the fact that the coordinator's final decision may not be available for some time (or actually lost).

Alternative

Let a participant P in the READY state timeout when it hasn't received the coordinator's decision; P tries to find out what other participants know (as discussed).

Observation

Essence of the problem is that a recovering participant cannot make a local decision: it is dependent on other (possibly failed) processes

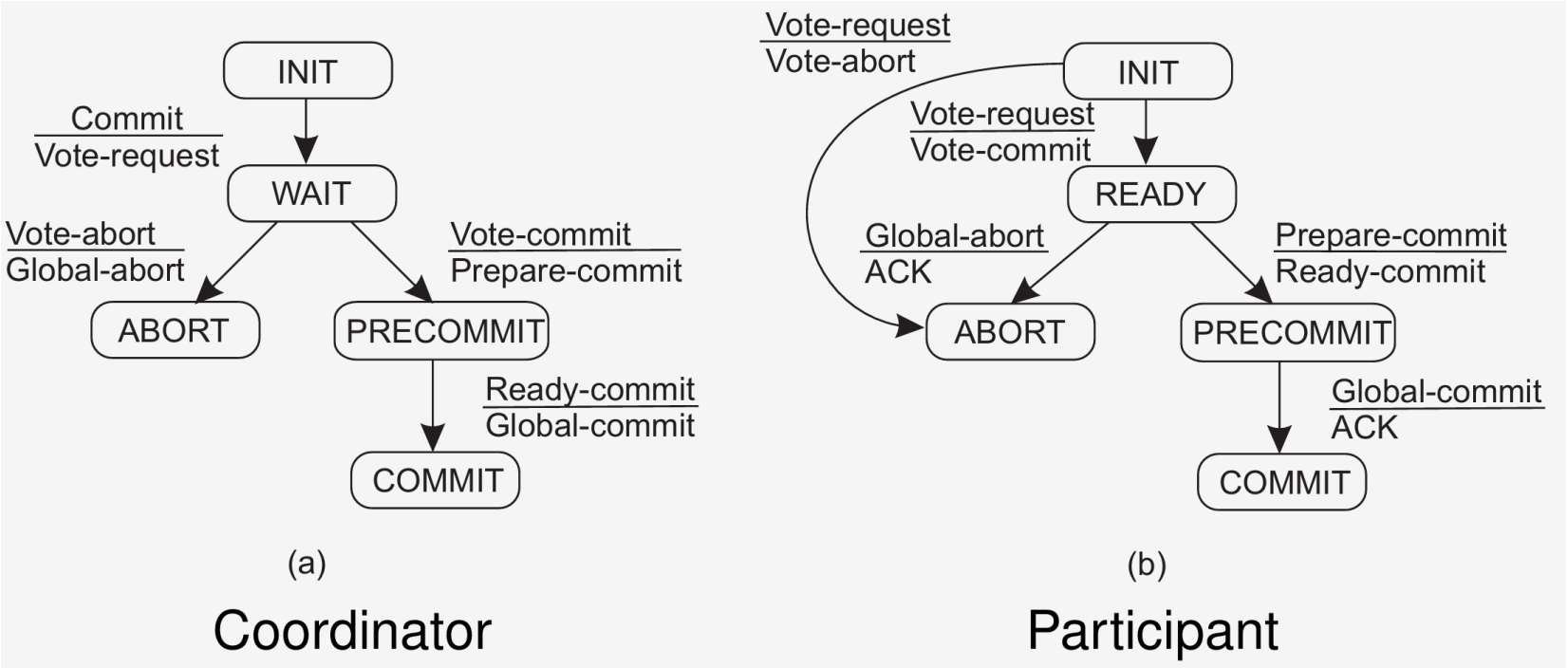
可靠的通信
Distributed Commit

Three-phase commit

Model (Again: the client acts as coordinator)
Phase 1a: Coordinator sends vote-request to participants
Phase 1b: When participant receives vote-request it returns either vote-commit or vote-abort to coordinator. If it sends vote-abort, it aborts its local computation
Phase 2a: Coordinator collects all votes; if all are vote-commit, it sends prepare-commit to all participants, otherwise it sends global-abort, and halts
Phase 2b: Each participant waits for prepare-commit, or waits for global-abort after which it halts
Phase 3a: (Prepare to commit) Coordinator waits until all participants have sent ready-commit, and then sends global-commit to all
Phase 3b: (Prepare to commit) Participant waits for global-commit

分布式提交
Distributed Commit

Three-phase commit



分布式提交
Distributed Commit

3PC– Failing participant

Basic issue

Can P find out what it should it do after crashing in the ready or pre-commit state, even if other participants or the coordinator failed?

Reasoning

Essence: Coordinator and participants on their way to commit, never differ by more than one state transition

Consequence: If a participant timeouts in ready state, it can find out at the coordinator or other participants whether it should abort, or enter pre-commit state

Observation: If a participant already made it to the pre-commit state, it can always safely commit (but is not allowed to do so for the sake of failing other processes)

Observation: We may need to elect another coordinator to send off the final COMMIT

恢复
Recovery

- Introduction
- Checkpointing
- Message Logging

- 简介
- 检查点设置
- 消息记录

恢复
Recovery

Recovery: Background

When a failure occurs, we need to bring the system into an error-free state:

1. Forward error recovery: Find a new state from which the system can continue operation
2. Backward error recovery: Bring the system back into a previous error-free state

Practice

Use backward error recovery, requiring that we establish recovery points

Observation

Recovery in distributed systems is complicated by the fact that processes need to cooperate in identifying a consistent state from where to recover

恢复
Recovery

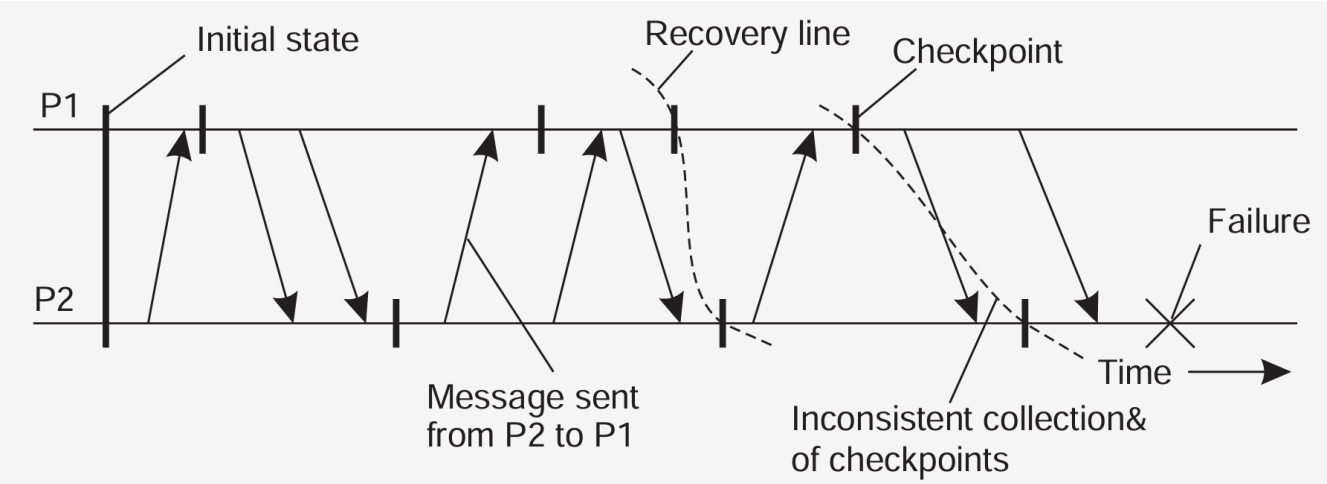
Consistent recovery state

Requirement

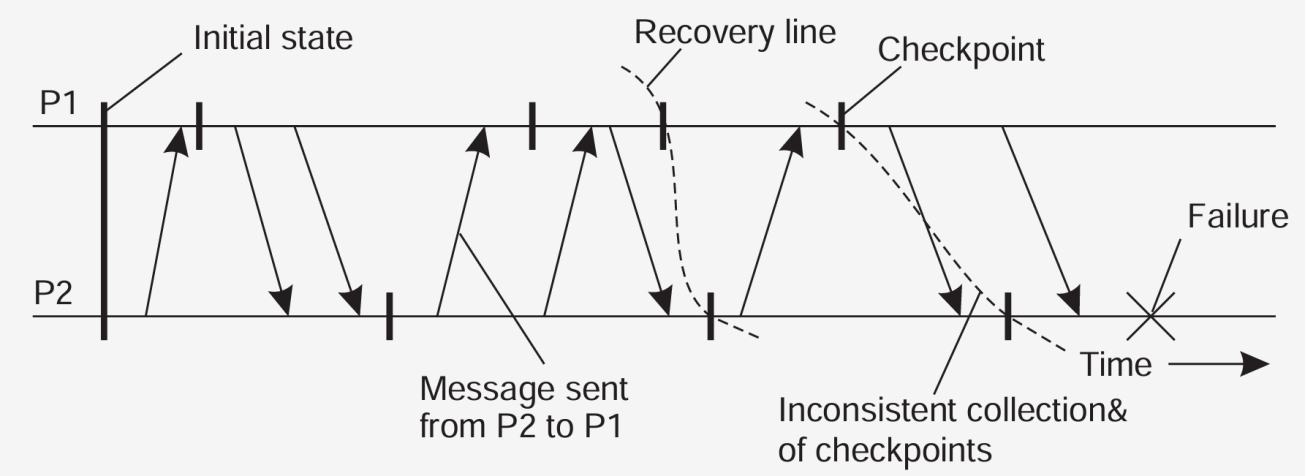
Every message that has been received is also shown to have been sent in the state of the sender.

Recovery line

Assuming processes regularly checkpoint their state, the most recent consistent global checkpoint.



恢复
Recovery



Observation

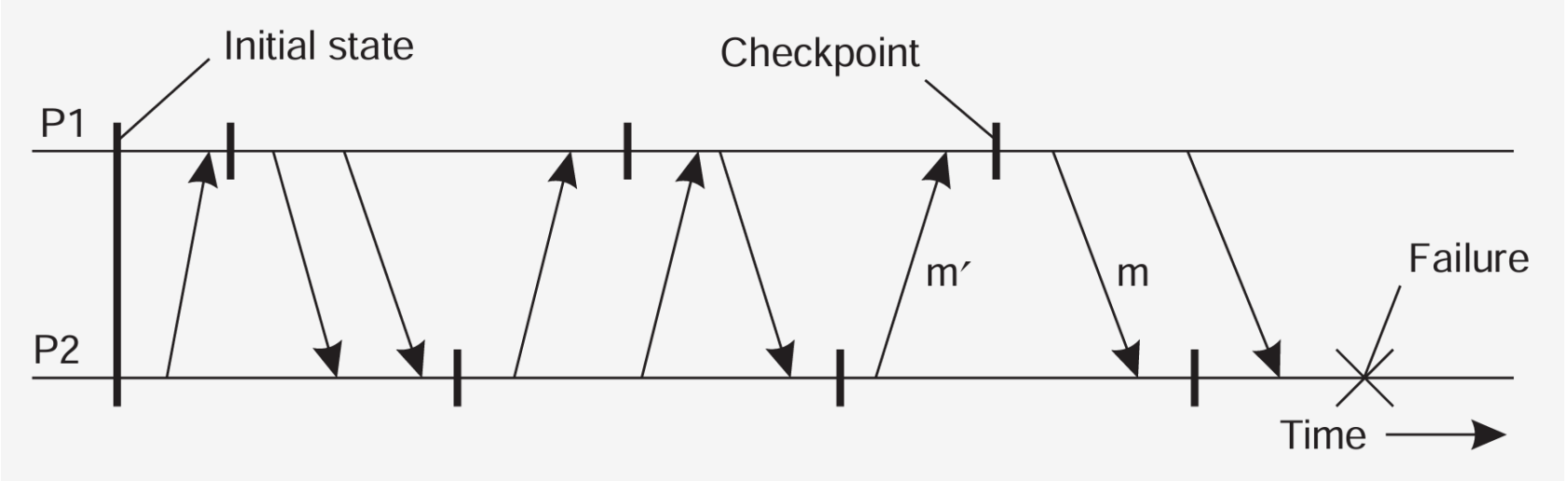
If and only if the system provides reliable communication, should sent messages also be received in a consistent state.

恢复
Recovery

Cascaded rollback

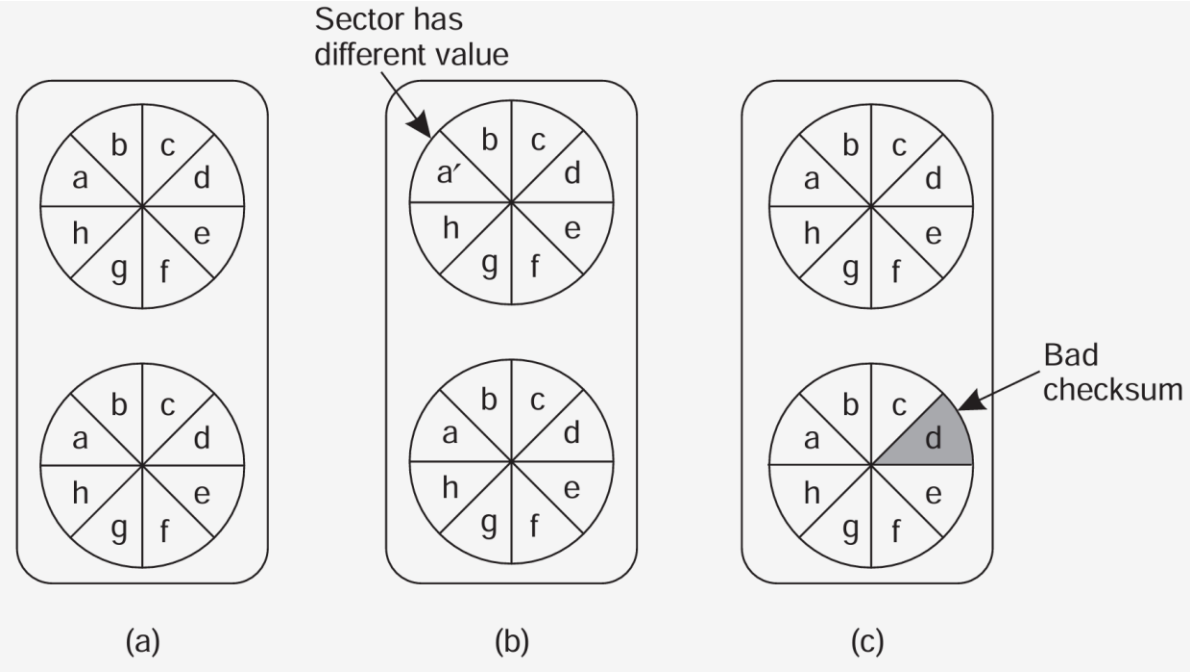
Observation

If checkpointing is done at the “wrong” instants, the recovery line may lie at system startup time ⇒ cascaded rollback



恢复
Recovery

Checkpointing: Stable storage



After a crash

- If both disks are identical: you're in good shape.
- If one is bad, but the other is okay (checksums): choose the good one.
- If both seem okay, but are different: choose the main disk.
- If both aren't good: you're not in a good shape.

恢复
Recovery

Independent checkpointing

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

- Let CP[i](m) denote mth checkpoint of process Pi and INT[i](m) the interval between CP[i](m−1) and CP[i](m)
- When process Pi sends a message in interval INT[i](m), it piggybacks (i,m)
- When process Pj receives a message in interval INT[j](n), it records the dependency INT[i](m) → INT[j](n)
- The dependency INT[i](m) →INT[j](n) is saved to stable storage when taking checkpoint CP[j](n)

恢复
Recovery

Independent checkpointing

Observation

If process Pi rolls back to CP[i](m−1), Pj must roll back to CP[j](n−1).

Question

How can Pj find out where to roll back to?

恢复
Recovery

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Question

What advantages are there to coordinated checkpointing?

恢复
Recovery

Coordinated checkpointing

Simple solution

Use a two-phase blocking protocol:

- A coordinator multicasts a checkpoint request message
- When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint
- When all checkpoints have been confirmed at the coordinator, the latter broadcasts a checkpoint done message to allow all processes to continue

Observation

It is possible to consider only those processes that depend on the recovery of the coordinator, and ignore the rest

恢复
Recovery

Message logging

Alternative

Instead of taking an (expensive) checkpoint, try to replay your (communication) behavior from the most recent checkpoint ⇒ store messages in a log.

Assumption

We assume a piecewise deterministic execution model:

- The execution of each process can be considered as a sequence of state intervals
- Each state interval starts with a nondeterministic event (e.g., message receipt)
- Execution in a state interval is deterministic

恢复
Recovery

Message logging

Conclusion

If we record nondeterministic events (to replay them later), we obtain a deterministic execution model that will allow us to do a complete replay.

Question

Why is logging only messages not enough?

Question

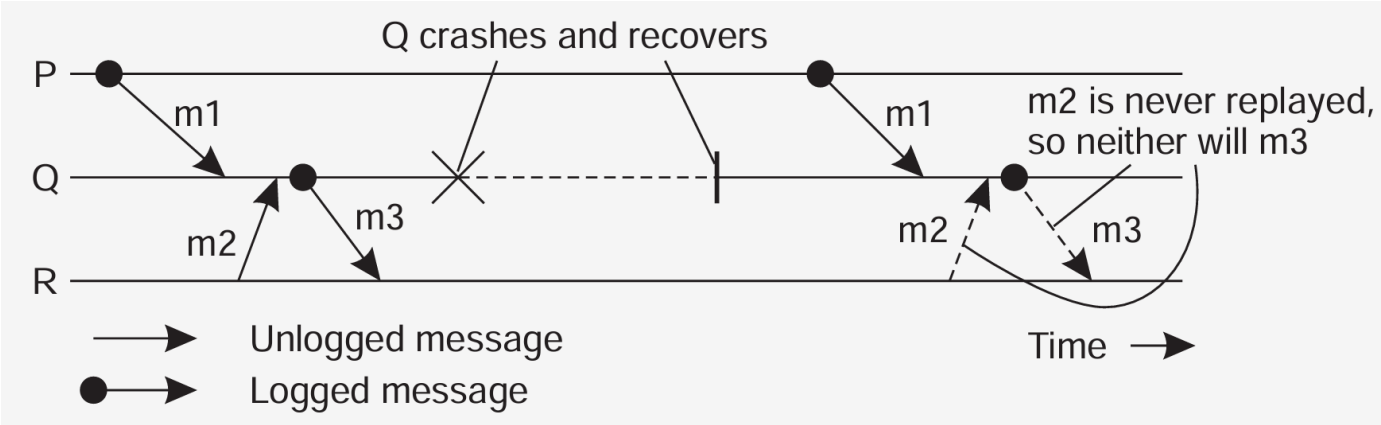
Is logging only nondeterministic events enough?

恢复
Recovery

Message logging and consistency

When should we actually log messages?

- Process Q has just received and subsequently delivered messages m1 and m2
- Assume that m2 is never logged.
- After delivering m1 and m2, Q sends message m3 to process R
- Process R receives and subsequently delivers m3



恢复
Recovery

Message-logging schemes

Notations

HDR[m]: The header of message m containing its source, destination, sequence number, and delivery number

The header contains all information for resending a message and delivering it in the correct order (assume data is reproduced by the application)

A message m is stable if HDR[m] cannot be lost (e.g., because it has been written to stable storage)

DEP[m]: The set of processes to which message m has been delivered, as well as any message that causally depends on delivery of m

COPY[m]: The set of processes that have a copy of HDR[m] in their volatile memory

恢复
Recovery

Message-logging schemes
Characterization
If C is a collection of crashed processes, then Q ∈ C is an orphan if there is a message m such that Q ∈DEP[m] and COPY[m]⊆C

Note
Wewant ∀m∀C ::COPY[m]⊆C ⇒DEP[m]⊆C. This is the same as saying that ∀m :: DEP[m] ⊆COPY[m].

Goal
No orphans means that for each message m, DEP[m]⊆COPY[m]

恢复
Recovery

Message-logging schemes

Pessimistic protocol

For each nonstable message m, there is at most one process dependent on m, that is |DEP[m]| ≤1.

Consequence

An unstable message in a pessimistic protocol must be made stable before sending a next message.

恢复
Recovery

Message-logging schemes

Optimistic protocol

For each unstable message m, we ensure that if COPY[m]⊆C, then eventually also DEP[m] ⊆ C, where C denotes a set of processes that have been marked as faulty

Consequence

To guarantee that DEP[m] ⊆C, we generally rollback each orphan process Q until Q ∈ DEP[m]

项目交付物/截止日期
Project Deliverables/Deadlines

Midterm(Survey/Review Paper):

Addressing Reviews and Submission Deadline: Thursday, June 4, 2026
Plagiarism Report: Friday, June 5, 2026
Final Submission Deadline: Thursday, June 11, 2026

Final Term(Technical/Research Contribution/Paper/Report):

Addressing Reviews and Submission Deadline: Thursday, June 25, 2026
Plagiarism Report: Friday, June 26, 2026
Final Submission Deadline: Thursday, July 02, 2026

项目交付物/截止日期
Project Deliverables/Deadlines

Midterm(Survey/Review Paper):

1. **Title & Authors:** concise, informative.
2. **Abstract:** 150–250 words — aim, methods, main results, key insights.
3. **Keywords:** 4–6 terms.
4. **Introduction:** context, motivation, research questions/objectives.
5. **Related Work / Background:** short review situating topic and bibliometric approach and existing review/survey papers.
6. **Data & Search Strategy:** data source(s) (e.g., Web of Science/Scopus), date range, search strings, inclusion/exclusion, record counts. (Include full search string in Appendix.)
7. **Preprocessing & Tools:** describe cleaning, deduplication, and software used (bibliometrix/BibShiny, VOSviewer) plus versions and key parameters.

项目交付物/截止日期
Project Deliverables/Deadlines

8. **Methods / Analysis Protocol:** outline metrics and analyses performed — descriptive stats (annual growth, sources, authors, countries), citation analysis, co-citation, bibliographic coupling, co-authorship, keyword co-occurrence, thematic mapping, trend analysis; network construction thresholds; visualization choices.
9. **Results:** structured by analysis type; include tables and readable figures (maps, networks, trend plots). Provide concise interpretation for each result.
10. **Discussion:** synthesize findings, compare with prior work, highlight thematic clusters and research gaps.
11. **Conclusions & Future Work:** main takeaways and suggested directions.
12. **Data & Code Availability / Reproducibility Statement:** link to scripts, datasets, and instructions for reproducing key figures.
13. **References**
14. **Appendices:** raw search strings, additional tables/figures, parameter settings, short README for code.

项目交付物/截止日期
Project Deliverables/Deadlines

- ❑ Search Strategy & Data (5 marks)
- ❑ Methodology & Analysis (5 marks)
- ❑ Use of Tools & Visualizations (5 marks)
- ❑ Results & Insights (5 marks)
- ❑ Writing, Structure & References (5 marks)
- ❑ Submission to the journal (5 marks)